Backend Engineer Roadmap (Node.js): Checklist

A step-by-step checklist from zero to interview-ready SDE / Backend Engineer. Tick every box before you pass the gate of a module.

How to use this

- Go in order. Each module has four parts: **Why it matters**, **Do**, **Build**, and **Move on when**.
- **Do** items are things to learn and practise. **Build** is a small hands-on task, because reading alone does not stick.
- **Move on when** is the exit gate. If you cannot do every item in the gate without notes, stay in the module.
- Run the **DSA track** (4-5 problems a week) from Module 2 onward, alongside everything else.
- Rough timeline: 6-8 months at 2-3 focused hours a day.

Module 0: Prerequisites (1 week)

Why it matters: Servers run on Linux, code lives in Git, and every concept later assumes you know how a request travels.

Do

- [ ] Linux terminal: navigate folders, read and edit files, permissions, processes (`ps`, `kill`), environment variables, `grep`, `curl`, SSH
- [ ] Git: commit, branch, merge, rebase, resolve a conflict, open a pull request
- [ ] How the web works: URL typed, DNS lookup, TCP connection, TLS, HTTP request, server, response, browser renders
- [ ] Client vs server: a request is just data (method, URL, headers, body), and a server is a program that waits for requests and replies with data

Build

- [ ] Push a small repo to GitHub using branches and a PR
- [ ] Use `curl` to call a public API and read the raw response headers

Move on when

- [ ] I can explain what happens from pressing Enter on a URL to seeing the page

- [ ] I can resolve a Git merge conflict without panic

Module 1: JavaScript Core (2 weeks)

Why it matters: Most backend bugs in JS come from a few misunderstood ideas, not from syntax. Interviews test these directly.

Do

- [ ] Variables, scope, hoisting, and the temporal dead zone (`var` vs `let` vs `const`)
- [ ] Value vs reference: why changing an object inside a function changes the caller's object
- [ ] Closures: a function remembers the variables from where it was created
- [ ] `this` and how it depends on how a function is called (`call`, `apply`, `bind`, arrow functions)
- [ ] Prototypes and classes: how objects inherit from other objects
- [ ] ES6+: destructuring, spread/rest, modules (CommonJS vs ES modules), Map/Set, optional chaining
- [ ] Array methods (`map`, `filter`, `reduce`) and immutability

Build

- [ ] Write `debounce`, `memoize`, and a deep clone from scratch
- [ ] Re-implement `Array.prototype.map` and `bind`

Move on when

- [ ] I can predict the output of closure and `this` puzzles before running them
- [ ] I can explain why a function mutated the caller's object, and how to avoid it

---

Module 2: Asynchronous JavaScript and the Event Loop (2 weeks)

Why it matters: A database call takes milliseconds, which is an eternity for a CPU. All of Node is built on handling that wait well. This is the single most-asked Node interview topic.

Do

- [ ] Why async exists: waiting should not freeze everything else
- [ ] Callbacks, callback hell, and the problem it caused
- [ ] Promises: states, chaining, `Promise.all`, `allSettled`, `race`, `any`

- [ ] `async` / `await` and how it maps onto promises
- [ ] Error handling in async code: `try/catch`, unhandled rejections, a forgotten `await`
- [ ] The event loop: call stack, task queues, microtasks (promises) vs macrotasks (timers)
- [ ] Concurrency vs parallelism

Build

- [ ] Write a `Promise` implementation (simplified) and a `retry(fn, times)` helper
- [ ] Write 5 snippets mixing `setTimeout`, promises, and `process.nextTick`, and predict their order

Move on when

- [ ] I can explain why `setTimeout(fn, 0)` runs after a resolved promise
- [ ] I can draw the event loop on paper and walk a snippet through it
- [ ] I can explain what happens if a promise rejects and nobody catches it

---

Module 3: TypeScript (1 week)

Why it matters: Most company backends use TypeScript. It catches mistakes before the code runs.

Do

- [ ] Types, interfaces, unions, enums, generics
- [ ] Type narrowing and utility types (`Partial`, `Pick`, `Omit`, `Record`)
- [ ] `tsconfig` basics and typing async functions

Build

- [ ] Convert one of your Module 1-2 exercises to TypeScript with strict mode on

Move on when

- [ ] I can type a function that takes and returns generic data
- [ ] I know the difference between `any`, `unknown`, and `never`

Module 4: Node.js Core (3 weeks)

Why it matters: You must know why Node can serve thousands of users on one thread, and exactly when it fails. "Why is my server frozen?" is a classic interview question.

Do

- [ ] Node runtime: V8, libuv, the event loop phases, the thread pool

- [ ] Non-blocking I/O: why waiting does not block other users
- [ ] Where Node breaks: CPU-heavy work blocks the event loop for everyone
- [ ] Fixes: worker threads, the cluster module, `child_process`, moving work to another service
- [ ] Built-in modules: `fs`, `path`, `events` (EventEmitter), `http`, `crypto`, `os`
- [ ] Streams and Buffers: sending a 2GB file in chunks, and backpressure (a fast producer slowing for a slow consumer)
- [ ] Memory: how garbage collection works, common leaks (forgotten listeners, growing global arrays)
- [ ] Errors: operational errors (DB down) vs programmer errors (bugs), crashing and restarting, graceful shutdown
- [ ] `package.json`, lockfiles, semantic versioning, `.env` configuration

Build

- [ ] Build an HTTP server using only the `http` module (no Express)
- [ ] Stream a large file to a client and watch memory stay flat
- [ ] Block the event loop with a heavy loop, prove it with two parallel `curl` calls, then fix it with a worker thread

Move on when

- [ ] I can explain why Node handles 10,000 connections but chokes on one heavy loop
- [ ] I can describe graceful shutdown and implement it
- [ ] I can find a memory leak using a heap snapshot or `--inspect`

Module 5: Networking and HTTP (2 weeks)

Why it matters: Your server and client talk across an unreliable, public network. Almost every production issue involves the network somewhere.

Do

- [ ] OSI and TCP/IP layers: what each does
- [ ] TCP (handshake, ordering, retransmission) vs UDP (speed, no guarantees)
- [ ] DNS: how a name becomes an IP, and caching
- [ ] HTTP: methods and their meaning, safe vs idempotent, status codes (2xx/3xx/4xx/5xx), headers, statelessness
- [ ] Cookies, caching headers (`Cache-Control`, `ETag`), content negotiation, keep-alive
- [ ] HTTPS/TLS: why public-key and symmetric encryption are both used, and what certificates prove

- [ ] HTTP/1.1 vs HTTP/2 vs HTTP/3
- [ ] CORS: a browser rule, how preflight works, and how to configure it
- [ ] Forward proxy, reverse proxy, load balancer, CDN
- [ ] Real-time: polling, WebSockets, Server-Sent Events, and when to use each
- [ ] gRPC and GraphQL: what problem each solves compared to REST

Build

- [ ] Inspect a real request in the browser Network tab and explain every header
- [ ] Build a small WebSocket chat and an SSE endpoint

Move on when

- [ ] I can explain 401 vs 403, PUT vs PATCH, and why GET must be safe
- [ ] I can explain a TLS handshake in simple steps
- [ ] I can explain why CORS errors happen and why Postman does not show them

---

Module 6: Building APIs with Express (2 weeks)

Why it matters: This is where you turn knowledge into working backends. The middleware pipeline is the heart of most Node frameworks.

Do

- [ ] Routing, request and response objects, params, query, body
- [ ] Middleware: a chain of functions where any step can stop the request
- [ ] Error-handling middleware and a central error format
- [ ] Validation at the edge with Zod or Joi (never trust incoming data)
- [ ] Environment config, structured logging, file uploads

Build

- [ ] A CRUD API (in memory first) with validation, logging middleware, and one global error handler

Move on when

- [ ] I can explain the request lifecycle through middleware without notes
- [ ] Every endpoint of mine validates input and returns consistent errors

---

Module 7: REST API Design (1-2 weeks)

Why it matters: Your frontend and other teams depend on your API. A bad design hurts everyone and is hard to change later.

Do

- [ ] Resource naming, correct methods and status codes, consistent error responses
- [ ] Versioning so old clients do not break
- [ ] Pagination: offset vs cursor, and why offset breaks at scale
- [ ] Filtering and sorting
- [ ] Idempotency: stopping a double charge when a client retries
- [ ] Rate limiting: fixed window, sliding window, token bucket
- [ ] OpenAPI/Swagger documentation
- [ ] NestJS: modules, controllers, providers, dependency injection, guards, pipes

Build

- [ ] Add cursor pagination and an idempotency-key mechanism to your CRUD API
- [ ] Implement a token-bucket rate limiter yourself
- [ ] Rebuild a small piece in NestJS

Move on when

- [ ] I can design the API set for a food-delivery app and defend each choice
- [ ] I can explain how my API behaves when a client retries a payment

---

Module 8: Relational Databases and SQL (3 weeks)

Why it matters: Data in a poor structure becomes the bottleneck of the whole system. Databases are the largest part of backend interviews.

Do

- [ ] Why a database instead of files: concurrency, crash safety, fast search
- [ ] PostgreSQL setup and basic queries
- [ ] Joins (inner, left, right), grouping, subqueries, CTEs, window functions
- [ ] Relationships: one-to-one, one-to-many, many-to-many (join tables)
- [ ] Keys and constraints: primary, foreign, unique, not null
- [ ] Normalization (1NF to BCNF) and deliberate denormalization
- [ ] Migrations: changing the schema while the app is live

Build

- [ ] Design and create a schema for an e-commerce app (users, products, orders, payments)
- [ ] Write 15 queries, including a top-N per group using window functions

Move on when

- [ ] I can design a normalized schema from a requirement and explain each table
- [ ] I can explain when denormalizing is the right call

Module 9: Indexes and Query Performance (1-2 weeks)

Why it matters: Scanning 10 million rows to find one is too slow. Indexes are the biggest single lever for database speed.

Do

- [ ] Why sorted data and binary search are fast, and how a B-tree extends that idea to disks
- [ ] Index types: single, composite (column order matters), unique, partial, covering
- [ ] The cost: faster reads, slower writes, more space
- [ ] Reading `EXPLAIN` / `EXPLAIN ANALYZE`: sequential scan vs index scan
- [ ] Connection pooling: why you reuse a small set of connections
- [ ] The N+1 problem with ORMs
- [ ] ORMs and query builders (Prisma, TypeORM, Knex): what they hide

Build

- [ ] Load 1 million rows, run a slow query, add an index, and compare `EXPLAIN` output before and after
- [ ] Reproduce an N+1 problem in an ORM, then fix it

Move on when

- [ ] I can explain why an index made a query 100x faster but slowed inserts
- [ ] I can pick the right composite index for a given query
- [ ] I can spot N+1 in code

Module 10: Transactions and Concurrency (2 weeks)

Why it matters: Two users buying the last item at the same moment is a real production scenario. Wrong answers here lose money.

Do

- [ ] ACID in plain words: all-or-nothing, consistency, isolation, durability
- [ ] Transactions: money transfer, where the server dies halfway
- [ ] Isolation problems: dirty read, non-repeatable read, phantom read, and which isolation level prevents each
- [ ] Locking: pessimistic (lock first) vs optimistic (version number check)
- [ ] MVCC at a basic level
- [ ] Deadlocks: how they happen and how to avoid them

Build

- [ ] Write a money-transfer function in a transaction and force a failure in the middle
- [ ] Simulate two simultaneous purchases of the last item, show the bug, then fix it two ways (pessimistic and optimistic)

Move on when

- [ ] I can explain what happens when two users buy the last item and how I prevent a wrong result
- [ ] I can explain each isolation level with an example

Module 11: NoSQL and Scaling Databases (2 weeks)

Why it matters: Not all data fits tables, and one machine cannot hold everything forever. You must know why each database type exists to choose well.

Do

- [ ] Document (MongoDB): flexible nested data, embedding vs referencing
- [ ] Key-value (Redis, DynamoDB), wide-column (Cassandra), graph, search (Elasticsearch): what pain each solves
- [ ] Replication: leader-follower, replication lag, reading from replicas
- [ ] Sharding/partitioning, choosing a shard key, hot shards
- [ ] Consistent hashing: adding a machine without reshuffling everything
- [ ] CAP theorem in plain words, ACID vs BASE, eventual consistency
- [ ] Choosing a database from the data and access pattern, not from popularity

Build

- [ ] Model the same blog app in Postgres and in MongoDB, and write down the trade-offs
- [ ] Implement consistent hashing in code

Move on when

- [ ] I can explain why not MongoDB for everything or Postgres for everything
- [ ] I can explain what breaks when you read from a replica right after a write

---

Module 12: Authentication and Authorization (2 weeks)

Why it matters: HTTP is stateless, so the server must be told who you are on every request. Mistakes here are security breaches.

Do

- [ ] Password storage: hashing, salting, why bcrypt/argon2 are deliberately slow
- [ ] Sessions (state on server) vs tokens (state in the token): trade-offs
- [ ] JWT: structure, signing, short-lived access tokens, refresh tokens, revocation, storage pitfalls
- [ ] OAuth 2.0 and "Login with Google": the flow step by step
- [ ] MFA, OTP, API keys
- [ ] Authorization: roles (RBAC), permissions, and checking ownership, not just role

Build

- [ ] Signup, login, refresh, and logout with hashed passwords and JWT
- [ ] Add role checks plus an ownership check (user A must not read user B's order)
- [ ] Add "Login with Google" using OAuth

Move on when

- [ ] I can explain why a JWT cannot simply be logged out, and how to work around it
- [ ] I can explain the OAuth flow and who talks to whom

Module 13: Caching and Performance (2 weeks)

Why it matters: The same expensive query running a thousand times for the same answer wastes money and time.

Do

- [ ] Where to cache: browser, CDN, application, database
- [ ] Patterns: cache-aside, write-through, write-back
- [ ] Staleness: TTL, invalidation, eviction (LRU/LFU)
- [ ] Cache stampede and how to prevent it
- [ ] Redis: strings, hashes, lists, sets, sorted sets, expiry, persistence, pub/sub
- [ ] Latency vs throughput, profiling before optimizing, load testing (k6, Artillery)

Build

- [ ] Add Redis cache-aside to a slow endpoint and measure before and after
- [ ] Build a leaderboard with a sorted set
- [ ] Load test an endpoint and find its breaking point

Move on when

- [ ] I can explain how I would cache a product page and what goes wrong when the price changes
- [ ] I can explain cache stampede and a fix

Module 14: Scaling and Surviving Growth (2 weeks)

Why it matters: As users and data grow, a single server will crash. This is the core of how real systems are built.

Do

- [ ] Vertical vs horizontal scaling and their limits
- [ ] Stateless servers: why sessions and uploads must not live in one server's memory
- [ ] Load balancing: round robin, least connections, health checks
- [ ] Queues as a buffer between fast producers and slow workers
- [ ] Parallelism: many workers reading one queue
- [ ] Nginx as reverse proxy and load balancer

Build

- [ ] Run 3 copies of your API behind Nginx, kill one, and confirm traffic continues

Move on when

- [ ] I can explain what breaks first as traffic goes up 10x, then 100x, and what I do at each step

---

Module 15: Background Jobs and Messaging (3 weeks)

Why it matters: Sending an email or generating a report takes seconds. Users should not wait, and a crash should not lose the work.

Do

- [ ] Background jobs: accept, enqueue, reply, process later
- [ ] Retries, backoff, dead-letter queues, scheduled jobs

- [ ] Queue (RabbitMQ, SQS) vs log (Kafka): partitions, consumer groups, offsets, replay
- [ ] Delivery guarantees: at-most-once, at-least-once, exactly-once
- [ ] Idempotent consumers: handling duplicates
- [ ] Event-driven design and pub/sub
- [ ] Scaling WebSockets across multiple servers

Build

- [ ] Email-sending worker with BullMQ, retries, and a dead-letter queue
- [ ] A small Kafka or RabbitMQ producer and consumer

Move on when

- [ ] I can explain what happens when a worker crashes halfway through a job
- [ ] I can explain why consumers must handle duplicates

Module 16: Reliability and Distributed Systems (2 weeks)

Why it matters: In a distributed system, networks drop, services slow down, and machines die. Good engineers design for failure.

Do

- [ ] Timeouts, retries with backoff and jitter, retry storms
- [ ] Circuit breaker, bulkhead, graceful degradation
- [ ] Distributed transactions: saga pattern and outbox pattern, and why two-phase commit is rarely used
- [ ] Monolith vs modular monolith vs microservices, and when each fits
- [ ] API gateway and service discovery basics

Build

- [ ] Add a timeout, retry with backoff, and a circuit breaker around a flaky external call

Move on when

- [ ] I can answer: payment succeeded but order creation failed, what now?
- [ ] I can explain why most teams should start with a well-structured monolith

---

Module 17: Security (2 weeks)

Why it matters: Anything on the internet will be attacked. Security flaws are the most expensive bugs.

Do

- [ ] SQL/NoSQL injection and parameterized queries
- [ ] XSS and CSRF
- [ ] Broken access control and IDOR
- [ ] SSRF and insecure deserialization
- [ ] Helmet and security headers, correct CORS, rate limiting
- [ ] Secrets management, dependency scanning, encryption in transit and at rest
- [ ] Safe logging (no passwords or tokens) and audit trails

Build

- [ ] Deliberately write a SQL-injectable endpoint, exploit it, then fix it
- [ ] Run a dependency audit and fix the findings

Move on when

- [ ] For any endpoint, I can list how an attacker might abuse it
- [ ] I can explain IDOR with an example and how to prevent it

---

Module 18: Code Design and Testing (3 weeks)

Why it matters: Code that works but cannot be changed is a liability. Teams hire engineers whose code others can maintain.

Do

- [ ] Layering: controller, service, repository
- [ ] SOLID, DRY, KISS, YAGNI, composition vs inheritance
- [ ] Patterns, each learned through the problem it removes: Strategy, Factory, Observer, Adapter, Decorator, Repository, Singleton
- [ ] Clean/hexagonal architecture and DDD basics
- [ ] Testing: unit, integration (with a real DB), end-to-end; Jest/Vitest and Supertest
- [ ] Mocks, stubs, fakes, and why over-mocking proves nothing
- [ ] TDD basics and what coverage does and does not tell you

Build

- [ ] Refactor a messy route handler into controller, service, and repository layers
- [ ] Replace a large `if/else` with the Strategy pattern
- [ ] Write unit and integration tests for the API from Module 7

Move on when

- [ ] I can explain why SQL does not belong in a route handler
- [ ] I can explain how to test code that calls a payment gateway

---

Module 19: DevOps, Cloud, and Observability (3 weeks)

Why it matters: "It works on my machine" does not help users. You are expected to ship and debug your own code.

Do

- [ ] Docker: images vs containers, layers, volumes, networks, Docker Compose
- [ ] CI/CD with GitHub Actions: test, build, deploy
- [ ] Deployment strategies: rolling, blue-green, canary, and rollback
- [ ] AWS basics: EC2, S3, RDS, Lambda, SQS, IAM, VPC; managed vs self-hosted; serverless trade-offs
- [ ] Kubernetes (awareness only) and PM2
- [ ] Observability: structured logs, metrics (Prometheus/Grafana), traces (OpenTelemetry), alerts
- [ ] Health checks, SLI/SLO/SLA

Build

- [ ] Dockerize your API with Postgres and Redis using Compose
- [ ] CI pipeline that tests and deploys to an AWS instance behind Nginx
- [ ] Add logs, a metric, and a health check

Move on when

- [ ] I can deploy an app from scratch and roll it back
- [ ] I can answer: the API is slow in production, where do I look first?

Parallel Track A: DSA (start at Module 2, never stop)

Why it matters: Almost every SDE interview has coding rounds, regardless of the role.

Do

- [ ] Big-O analysis
- [ ] Arrays, strings, hashing, two pointers, sliding window
- [ ] Linked lists, stacks, queues
- [ ] Recursion, backtracking, binary search
- [ ] Trees, heaps, graphs (BFS, DFS, shortest path)

- [ ] Dynamic programming, tries, sorting

Move on when

- [ ] I can solve most easy and medium problems in 30-40 minutes, explaining my thinking aloud
- [ ] I have solved 200+ problems across all topics above

Parallel Track B: CS Fundamentals (alongside Modules 4-10)

Why it matters: Interviewers use OS and DBMS questions to test depth beyond frameworks.

Do

- [ ] OS: process vs thread, scheduling, memory and virtual memory, deadlocks, mutex and semaphore, I/O models
- [ ] Networks: revisit Module 5 at interview depth
- [ ] DBMS: revisit Modules 8-10 at interview depth

Move on when

- [ ] I can explain each topic in two minutes without notes

Module 20: System Design (start after Module 15)

Why it matters: You cannot copy another company's design. YouTube, Netflix, and Hotstar all stream video but need different designs because their traffic shape, budget, and predictability differ. Design is reasoning about trade-offs, not memorizing architectures.

Do

- [ ] Fundamentals: scalability, availability, consistency, latency, back-of-envelope estimation
- [ ] Always ask: traffic shape, read vs write ratio, consistency need, budget
- [ ] Low-level design: parking lot, BookMyShow, elevator system
- [ ] High-level design: URL shortener, chat, notification service, news feed, payment system

Build

- [ ] Write a 1-page design doc for 5 systems, including the trade-offs you rejected and why

Move on when

- [ ] I can design a new system in 45 minutes: requirements, estimates, components, bottlenecks, trade-offs
- [ ] I can say what would change if traffic were 100x or unpredictable

---

Module 21: Projects and Interview Readiness (ongoing)

Why it matters: Projects prove you can combine everything. Two deep projects beat ten shallow ones.

Do

- [ ] Project 1: production-style REST API with auth, RBAC, Postgres, Redis caching, tests, Docker, CI/CD
- [ ] Project 2: event-driven system (e-commerce or ride-hailing) with queues, workers, and retries
- [ ] Project 3: real-time chat with WebSockets scaled across instances
- [ ] Resume with measurable results, clean GitHub READMEs
- [ ] Mock interviews (at least 5) and behavioural stories

Move on when (ready to apply)

- [ ] I can defend every decision in my projects (why this DB, why this queue, what breaks at 100x)
- [ ] I can explain the event loop, an index, a transaction, and a JWT without notes
- [ ] I pass mock interviews on DSA, backend, and design consistently